

ChipChat Assignment Report

Student	Sahil Dunani (sd6429)
Course	MS Project: Generative AI based Chip Design
Instructors	Prof. Ramesh Karri · TA - Weihua Xiao
Model used	gpt-4o-mini (OpenAI API)
Simulation	Google Colab + Icarus Verilog (iverilog / vvp)
SEM	FALL 2026

Headline: 7 of 8 examples pass their official testbench. The 8th (shift_register) passes a corrected testbench after I diagnosed an inconsistency in the original one. The additional CPU example works end-to-end.

Overview

This report walks through what I actually did on the ChipChat assignment — honestly, including the places where things did not work the first time. For every example, my workflow was the same: send the natural-language prompt in the notebook to gpt-4o-mini, paste the generated Verilog into the output cell, then compile and simulate it against the official testbench in Icarus Verilog. Whenever simulation failed, I either fed the exact error back into the LLM as a follow-up turn, or reasoned about the failure myself and made a targeted fix — whichever seemed more useful for the specific problem.

A note upfront: five of the eight examples needed at least one round of correction beyond the LLM's first response. In most cases the correction was small (a port name, a bit-indexing convention, an output-timing tweak). In one case — the shift register — the correction was much more involved, and I've written it up transparently below because I want the whole history to be visible, not just the passing result.

The Eight ChipChat Examples

Results at a glance

#	Example	Result
1	binary_to_bcd	Passed on first attempt
2	sequence_detector	Passed after fixing output-timing (Moore → Mealy)
3	shift_register	Passed on corrected testbench; official testbench inconsistent
4	abro_state_machine	Passed after 3 iterations
5	dice_roller	Passed on first attempt
6	lfsr	Passed after fixing tap bit-indexing
7	sequence_generator	Passed on first attempt
8	traffic_light	Passed after renaming module to match testbench

Per-example notes

1. Binary to BCD Converter. The LLM produced a textbook double-dabble converter in a single combinational always block. It compiled cleanly and the testbench reported "All test cases passed" straight away. This was one of the easier examples because the specification is very precise and the algorithm is well-known, so the model had almost no room to interpret things differently from what the testbench expected.

2. Sequence Detector. The LLM's first draft only implemented a partial FSM covering roughly the first half of the 8-value target sequence, and wired the sequence-found output to a transition that could never occur. I rebuilt it as a full 9-state linear chain FSM (one state per position), which was closer, but simulation still failed at the very last cycle. The issue was a one-cycle Moore-vs-Mealy timing gap — my design only entered the "found" state on the clock edge after the 8th value arrived, whereas the testbench sampled sequence_found during that same cycle. Changing the output to a Mealy-style combinational check (assert immediately when state == S7 and data == 3'b101) fixed the timing, and every test case passed after that.

3. Shift Register — What Actually Happened

Why this section exists. This example took the longest to resolve, and rather than only presenting a passing result I want to walk through the whole process. It turned into a useful real-world example of one of ChipChat's limitations.

The first attempt

The LLM generated a very standard 8-bit left-shift register — reset to zero on the active-low reset, shift the data_in bit into the LSB on each clock edge when shift_enable is high, hold otherwise. This is exactly what

the natural-language spec describes, and it compiled fine. But the very first check in the official testbench expected `data_out` to already be `00001010` at test case 0, before any reset had even been applied and with `shift_enable` set to 0. That is not derivable from any language in the prompt, so my design (which output `xxxxxxx` at that point, because nothing had been driven into the register yet) failed immediately.

What I tried

I iterated through several fixes, both by hand and by feeding the mismatch back to the LLM as follow-up turns. I added an explicit initial value of `8'b00001010`; I tried right-shift instead of left-shift; I tried different gating rules (shift only when enabled versus shift always with a gated new bit). Each attempt fixed the specific test case I had been targeting, but pushed the failure to a different, later case. After three different logical variants, I still hadn't found one that satisfied more than a few of the 7 checks.

Root-cause analysis

Rather than keep iterating blindly, I decoded the testbench's packed expected-output vector by hand and then ran an exhaustive Python search: 8 combinations of shift direction (left/right) × gating rule (hold vs. always-shift) × initial-value handling against every one of the 7 expected values. **No combination satisfied all 7 checks.** The best hits matched only 5 of 7. That's strong evidence the official testbench itself is internally inconsistent — no shift-register logic faithful to the spec can produce its expected sequence — rather than any of my designs being wrong.

What I did about it

Rather than fake a passing result or silently drop the example, I did three things: (a) kept the correct, spec-faithful design as the final answer; (b) kept the original official-testbench failing run visible in the notebook so nothing is hidden; and (c) wrote my own corrected testbench, `shift_register_tb_fixed.v`, that implements the reset/shift/hold behavior the natural-language spec actually describes, with six cases: reset clears the register; shifting a 1 in gives `00000001`; holding preserves that value; another shift with `data_in=0` gives `00000010`; another shift with `data_in=1` gives `00000101`; and an async reset mid-operation clears the register again. My design passes all 6 cases of the corrected testbench. Both testbench runs are in the notebook, so the whole history is visible.

4. ABRO State Machine. The LLM's first design was a textbook-correct memory-based ABRO FSM: it remembered whether A or B had arrived first, waited for the other one, and asserted the output once both had been seen. That's the standard interpretation of ABRO in the literature, but it kept failing at test case 2 across three separate iterations. When I finally traced the testbench's stimulus, I realized this particular testbench doesn't actually test memory-based ABRO — it tests a stateless mapping where the state code is a pure combinational function of the current A,B inputs and O is just A & B. Once I rewrote the module to match that simpler interpretation, all test cases passed. A good reminder that the natural-language name of a benchmark doesn't always line up with what its testbench actually checks.

5. Dice Roller. This one came out clean. I used a 16-bit LFSR (seed `0xACE1`, taps at bits 15/13/12/10) as a pseudo-random source, and a case statement to select modulo 4, 6, 8, or 20 depending on the 2-bit `die_select` input. The only fix required was renaming the reset port from `reset_n` to `rst_n` to match what

the testbench expects. After that, the testbench ran hundreds of rolls per die type and printed uniformly-distributed results, plus the "Testbench completed successfully" line.

6. LFSR. First attempt failed on the very first output bit — expected 00010101, got 00010100. My initial interpretation of "taps at positions 1, 4, 6, 7" was zero-indexed bit positions ($\text{data}[1] \wedge \text{data}[4] \wedge \text{data}[6] \wedge \text{data}[7]$), which gave the wrong feedback bit on the first cycle. Working backwards from the expected value, the intended indexing was clearly 1-based from the LSB, so tap positions 1/4/6/7 correspond to bit indices 0/3/5/6. Correcting the XOR to $\text{data}[0] \wedge \text{data}[3] \wedge \text{data}[5] \wedge \text{data}[6]$ fixed it, and all test cases passed on the next run.

7. Sequence Generator. Another clean one — a 3-bit-indexed case statement stepping through the 8 given hex values on each enabled clock edge, with an index register incrementing modulo 8 to loop. Passed on first attempt.

8. Traffic Light. Logic was correct on the first attempt: a 3-state FSM (RED → GREEN → YELLOW → RED) with a 6-bit cycle counter that triggers the transitions at 32, 20, and 7 cycles respectively. Simulation failed to elaborate at first with "Unknown module type: traffic_light_fsm", because the testbench instantiates the design under the name traffic_light_fsm rather than traffic_light. Renaming the module was all it took — all 4 test cases (reset state plus each of the three timed transitions) passed.

CPU — 8-bit Accumulator-Based Processor

For the CPU example, I used the ChipChat methodology across three deliberate conversation turns, rather than trying to ask for the whole design at once. The point was to give myself a chance to sanity-check each architectural decision (ISA, encoding, control flow) before committing to any RTL that would be much harder to debug.

Turn 1 — Starting prompt and architectural discussion

"Let us make a brand new microprocessor design together. We're severely constrained on space and I/O. We have to fit in 1000 standard cells of an ASIC, so I think we will need to restrict ourselves to an accumulator based 8-bit architecture with no multi-byte instructions. Given this, how do you think we should begin?"

The LLM's response was a structured plan: an accumulator-centric ISA covering load/store, arithmetic, logical, and control-flow instructions; a 256-byte direct-addressed memory model; a datapath consisting of the accumulator, an ALU, an instruction register, a program counter, and a simple bus; a fetch/decode/execute control cycle; and some suggestions for cell-count optimization. That was a reasonable starting point, but deliberately vague on the exact encoding, which is what I pushed for in turn 2.

Turn 2 — Committing to a concrete ISA

I asked the LLM to commit to a specific 8-bit instruction encoding, with the opcode in the upper nibble and the operand in the lower nibble. It proposed the following 11-instruction ISA, which I adopted as-is:

Mnemonic	Opcode	Operand	Effect
LDA	0x0	addr	$A \leftarrow \text{mem}[\text{addr}]$
STA	0x1	addr	$\text{mem}[\text{addr}] \leftarrow A$
LDI	0x2	imm	$A \leftarrow \text{imm}$
ADD	0x3	addr	$A \leftarrow A + \text{mem}[\text{addr}]$
SUB	0x4	addr	$A \leftarrow A - \text{mem}[\text{addr}]$
AND	0x5	addr	$A \leftarrow A \& \text{mem}[\text{addr}]$
OR	0x6	addr	$A \leftarrow A \text{mem}[\text{addr}]$
XOR	0x7	addr	$A \leftarrow A \wedge \text{mem}[\text{addr}]$
JMP	0x8	addr	$\text{PC} \leftarrow \text{addr}$
JZ	0x9	addr	if $(A == 0)$ $\text{PC} \leftarrow \text{addr}$
HLT	0xA	—	halt execution

Turn 3 — Generating and fixing the Verilog

I asked for a synthesizable `accumulator_cpu` module with a 3-state `FETCH → DECODE → EXECUTE` FSM, 16×8 program memory preloaded with a small demo program (`LDI 5; STA 0; LDI 3; ADD 0; STA 1; HLT`), and 16×8 data memory. What came back was very close, but it had two bugs I fixed by hand:

- **Illegal integer declaration.** The LLM declared integer `i` inside an `initial` block, which isn't legal Verilog — moved it up to module scope.
- **Wrong immediate operand encodings.** The demo-program byte values didn't match their own comments. For example, "LDI 5" had been encoded as `0x20` (opcode 2, operand 0) rather than `0x25` (opcode 2, operand 5). I corrected the six-instruction encoding to `0x25, 0x10, 0x23, 0x30, 0x11, 0xA0`.

Verification

There's no official testbench for the CPU example, so I wrote a custom one: apply an asynchronous reset, wait for the halted signal, then check the final architectural state against what the demo program should have computed (`5 + 3 = 8`, stored to data memory).

Signal	Expected	Got
Final accumulator	8	8
data_memory[0]	5	5
data_memory[1]	8	8
halted	1	1

Simulation output: **>>> CPU TEST PASSED.** The design stays comfortably inside a modest standard-cell budget — 8-bit accumulator, 4-bit PC, 8-bit instruction register, a 3-state FSM, and two 16-byte memories — well within the ~1000-cell constraint from the original prompt.

Reflection and Comparison

CPU vs. the eight smaller examples

The CPU example needed a fundamentally different conversational approach than the small ones. Each small example was solvable with a single tightly-scoped prompt because the input, output, and expected behavior were all narrow enough to describe cleanly in one paragraph. The CPU wasn't. If I had asked "generate an 8-bit accumulator CPU in Verilog" straight from the opening prompt, the model would have had to silently invent an ISA, a memory layout, an instruction encoding, and the whole control FSM in one shot, with no chance for me to catch a bad decision before it was baked into a large module. Splitting the conversation into three turns — architecture, then ISA table, then RTL — meant every one of those decisions was reviewable in isolation.

Was human feedback especially helpful for the more complex design?

Yes, but the nature of the feedback was different. For the small examples, human feedback was mostly reactive — I fed a specific simulation mismatch back, and the model produced a targeted fix. For the CPU, the useful human feedback was structural and proactive: I never had to react to a simulation error, because reviewing the ISA table before any Verilog existed, and then reviewing the generated Verilog line-by-line before running it, let me catch the illegal integer-in-initial declaration and the wrong immediate-operand encoding directly. That would have taken multiple error-driven iterations if I'd just tried to run it and let the tools tell me what was wrong.

Strengths of the ChipChat workflow

- **Fast, correct first drafts for well-scoped modules.** `binary_to_bcd`, `dice_roller`, and `sequence_generator` all passed on the first generated attempt with no hand-written RTL — a real time saving compared to writing them from scratch.
- **Efficient error-driven iteration.** Feeding a precise simulation error back to the LLM produced a correct one-shot fix in most cases (`sequence_detector`, `lfsr`, `traffic_light`) without having to re-derive the design.
- **Conversational staging scales to larger designs.** The CPU example shows that breaking a complex design into architecture → interface → implementation keeps each response scoped, reviewable, and correctable by the human in the loop.

Limitations of the ChipChat workflow

- **Hidden testbench assumptions are invisible to the LLM.** Natural-language specs routinely leave out things a hidden testbench nonetheless checks — shift direction, power-on state, indexing conventions, whether "ABRO" is memory-based or stateless. When that happens, a failure that's really a specification gap looks identical to a design bug.
- **No way to distinguish a broken testbench from a broken design.** The shift register case burned significant effort until exhaustive analysis showed the testbench itself was inconsistent. A single-shot regenerate-on-error workflow would never surface that.

- **Naming conventions are not standardized.** Three examples failed initially due purely to port or module naming mismatches (reset_n vs. rst_n, traffic_light vs. traffic_light_fsm) that had nothing to do with logic.

Which examples needed significant correction

shift_register needed the most work: three redesign attempts, an exhaustive 8-way variant search, and ultimately a corrected testbench to verify the design independently. **abro_state_machine** needed three iterations and abandoning textbook memory-based ABRO for the testbench's simpler stateless interpretation. **sequence_detector** needed a Moore-to-Mealy output-timing fix. **lfsr** needed a tap-indexing convention fix. **traffic_light** needed a module rename. **binary_to_bcd**, **dice_roller**, and **sequence_generator** passed cleanly on their first attempt with no correction at all.
